A MINIMAL FIELD HANDBOOK

The Minimal PyTorch Handbook

From tensors and training loops to Transformers and reinforcement learning - the working set, not the reference.

Mosafer

PyTorch 2.x · Gymnasium · Hugging Face

V1.0 - COMPACT EDITION

OCTOBER 2026

Contents

1 Orientation **3**
- 1.1 What this book is . . . 3
- 1.2 Installation . . . 3
- 1.3 A first program . . . 3

2 Tensors **5**
- 2.1 Creating tensors . . . 5
- 2.2 Dtypes and devices . . . 5
- 2.3 Shape surgery . . . 6
- 2.4 Indexing . . . 6
- 2.5 Broadcasting . . . 6
- 2.6 Reductions and the meaning of dim . . . 7

3 Autograd **8**
- 3.1 The basic cycle . . . 8
- 3.2 Disabling the graph . . . 8
- 3.3 Freezing parameters . . . 9
- 3.4 Where the graph ends . . . 9

4 Building Models: nn.Module **10**
- 4.1 The anatomy of a module . . . 10
- 4.2 The layer catalog . . . 11
- 4.3 Containers . . . 11
- 4.4 Initialization . . . 11
- 4.5 Saving and loading . . . 12

5 Data Pipelines **13**
- 5.1 The Dataset protocol . . . 13
- 5.2 The DataLoader . . . 14
- 5.3 Transforms and augmentation . . . 14
- 5.4 Train/val/test splits . . . 15
- 5.5 Variable-length samples: collate_fn . . . 15

6 The Training Loop **16**
- 6.1 The canonical template . . . 16
- 6.2 Loss functions . . . 17
- 6.3 Optimizers . . . 17
- 6.4 Learning-rate schedules . . . 17
- 6.5 Gradient clipping and accumulation . . . 18
- 6.6 Mixed precision . . . 18
- 6.7 Measuring accuracy . . . 18
- 6.8 Early stopping . . . 19

7 Worked Example: CIFAR-10 **20**
- 7.1 Data . . . 20
- 7.2 A small ResNet . . . 20
- 7.3 Training configuration . . . 21

7.4 Transfer learning in five lines . . . 21

8 Sequence Models: the LSTM **23**
8.1 The recurrence interface . . . 23
8.2 Worked example: text classification . . . 23
8.3 Packed sequences: don't run on padding . . . 24
8.4 Stateful RNNs and truncated BPTT . . . 24

9 Transformers **26**
9.1 Attention from scratch . . . 26
9.2 The block, and stacking . . . 26
9.3 A tiny character-level GPT . . . 27
9.4 Fine-tuning with Hugging Face . . . 29

10 Reinforcement Learning **30**
10.1 The environment API: Gymnasium . . . 30
10.2 REINFORCE: policy gradient . . . 30
10.3 DQN: learning the value of actions . . . 31
10.4 Vectorized environments . . . 33
10.5 PPO, complete . . . 33
10.6 Continuous control . . . 35
10.7 Choosing an algorithm . . . 36

11 Experiments and Statistics **37**
11.1 The unit of evidence . . . 37
11.2 When is the improvement real? . . . 37
11.3 Ablation discipline . . . 38
11.4 Hyperparameter sweeps with Optuna . . . 38

12 Research Craft **40**
12.1 Reproducibility . . . 40
12.2 Experiment hygiene . . . 40
12.3 Logging . . . 41
12.4 Speed checklist . . . 41
12.5 Multi-GPU, complete . . . 41
12.6 Speed and memory . . . 42
12.7 The debugging checklist . . . 43
12.8 Where to go next . . . 44

A Cheat Sheet **45**
A.1 Tensors . . . 45
A.2 Autograd . . . 45
A.3 Models . . . 46
A.4 Data and training . . . 46
A.5 RL with Gymnasium . . . 46
A.6 Idioms worth memorizing . . . 47
A.7 The book at a glance . . . 47

CHAPTER 1

Orientation

1.1 What this book is

This is a working handbook, not a reference. It covers the smallest set of PyTorch that lets you train real models end to end: supervised networks, Transformers, and conventional reinforcement learning agents. Everything is built from canonical boilerplate—the patterns you will retype for the rest of a research career. It assumes you know Python; it does not assume you know deep learning frameworks.

The entire book stands on four abstractions:

1. `torch.Tensor` — a GPU-ready, multi-dimensional array (Ch. 2).
2. Autograd — automatic differentiation, the engine of learning (Ch. 3).
3. `nn.Module` — how models are declared and composed (Ch. 4–5).
4. The training loop — forward, loss, backward, step (Ch. 6).

Once the loop in Chapter 6 is familiar, every application is a variation: a CNN replaces the MLP, a Transformer replaces the CNN, an environment's returns replace the labels. Read linearly the first time; after that, the cheat sheet in Appendix A is the index you will actually use.

1.2 Installation

```
pip install torch torchvision          # CPU-only; enough for most of this book
pip install torch torchvision \
    --index-url https://download.pytorch.org/whl/cu121   # CUDA build, match your driver
pip install gymnasium[classic-control] transformers datasets optuna   # for Ch. 9-11
```

Verify the install and locate your hardware before writing any model:

```
import torch

print(torch.__version__)                # e.g. 2.4.0
print(torch.cuda.is_available())        # True if a usable GPU exists
device = "cuda" if torch.cuda.is_available() else "cpu"
print(torch.cuda.get_device_name(0) if device == "cuda" else "cpu only")
```

> **HARDWARE REALITY**
>
> Every example in this book trains on a laptop CPU in minutes, except the CIFAR-10 CNN (Ch. 7) and the tiny GPT (Ch. 9), which want a GPU for comfortable iteration. Nothing here requires a multi-GPU machine.

1.3 A first program

Thirty seconds of PyTorch, no training yet:

```python
import torch

a = torch.randn(2, 3)        # random 2x3 matrix, requires_grad=False
w = torch.randn(3, 2, requires_grad=True)   # a learnable parameter
y = a @ w                    # matrix multiply: (2, 3) @ (3, 2) -> (2, 2)
y.sum().backward()           # autograd computes d(sum)/dw
print(w.grad.shape)          # torch.Size([3, 2])
```

If this runs, autograd works, and you have the whole idea in miniature: build an expression from tensors, call `backward()`, read gradients. Everything that follows—models, optimizers, RL agents—is this pattern scaled up and organized.

> **PITFALL**
>
> Conventions used throughout: `B` batch size, `L` sequence length, `C` channels, `D` feature dimension. A batch of images has shape `(B, C, H, W)`; a batch of token sequences has shape `(B, L)`. Most beginner bugs are shape bugs; state the expected shape in a comment before you write the line that produces it.

CHAPTER 2

Tensors

A `torch.Tensor` is a multi-dimensional array with three properties you must keep straight at all times: its *shape*, its *dtype*, and its *device*. Almost every runtime error in PyTorch is one of these three being wrong.

2.1 Creating tensors

```
import torch

t = torch.tensor([[1.0, 2.0], [3.0, 4.0]])   # from data (copies)
z = torch.zeros(3, 4)                        # all zeros, float32
o = torch.ones(2, 3)
r = torch.rand(2, 3)                         # uniform in [0, 1)
g = torch.randn(2, 3)                        # standard normal - the default for weights
i = torch.randint(0, 10, (2, 3))             # ints in [0, 10) - high is EXCLUSIVE
a = torch.arange(0, 10, 2)                   # [0, 2, 4, 6, 8]
l = torch.linspace(0, 1, 5)                  # 5 points in [0, 1]
e = torch.empty(2, 3)                        # uninitialized - fill before use
full = torch.full((2, 3), 7.0)
```

Factory functions that mirror an existing tensor's shape and device are the ones you will use most in real code:

```
x = torch.randn(4, 8)
mask  = torch.zeros_like(x)      # same shape/dtype/device as x
noise = torch.randn_like(x)
```

2.2 Dtypes and devices

The default dtype is `float32`; class labels must be `int64` for the classification losses. Convert with `.to()`, or with the shortcuts `.long()`, `.float()`, `.half()`:

```
w = torch.randn(3, 3, dtype=torch.float64)
x = torch.randn(2, 2, requires_grad=True)
labels = torch.tensor([0, 2, 1])        # dtype int64, ready for CrossEntropyLoss

x64 = x.to(torch.float64)
xcpu = x64.to("cpu")                    # device + dtype in one call
xgpu = xcpu.to("cuda", torch.float32)

n = x.sum().item()                      # tensor -> Python number
```

PITFALL

Two tensors must sit on the same device to interact; a CPU tensor and a CUDA tensor in one expression raises `RuntimeError`. Cast explicitly at the boundary: `x = x.to(device)` right after loading, and move

targets too. Call `.item()` (or `.cpu()`) the moment you need a value for logging—never accumulate GPU tensors in a Python list across a long loop.

2.3 Shape surgery

```
x = torch.randn(2, 3, 4)
x.shape            # torch.Size([2, 3, 4]); also x.size()

x.view(2, 12)      # same memory, new shape - needs a contiguous tensor
x.reshape(2, 12)   # view if possible, copy otherwise - the safe default
x.flatten(1)       # (2, 12): flatten from dim 1 on; keeps the batch dim
x.unsqueeze(0)     # (1, 2, 3, 4): add a dim; x[None] does the same
x.squeeze()        # drop all size-1 dims
x.permute(2, 0, 1) # reorder dims: (4, 2, 3)
x.transpose(1, 2)  # swap two dims: (2, 4, 3) - standard for (B, L, D) <-> (B, D, L)
```

```
a = torch.randn(2, 3)
b = torch.randn(2, 3)
torch.cat([a, b], dim=0).shape    # (4, 3) - concatenate along an EXISTING dim
torch.stack([a, b], dim=0).shape  # (2, 3) per element -> (2, 2, 3): NEW dim
```

VIEW VS. RESHAPE

`view` is free (no copy) but requires the tensor to be *contiguous*—memory laid out row-major with no gaps. Operations like `transpose` or `permute` make tensors non-contiguous; a subsequent `view` then fails. `reshape` handles both cases, and `.contiguous()` repairs a tensor when an API insists on `view`.

2.4 Indexing

```
x = torch.randn(4, 5)
x[0]            # first row
x[:, 0]         # first column, shape (4,)
x[1:3, ::2]     # rows 1-2, every other column
x[..., -1]      # last element of the last dim

x[[0, 2], [1, 3]]          # fancy: elements (0,1) and (2,3)
x[x > 0]                   # boolean mask -> 1-D tensor of matches
x[x > 0] = 0.0             # in-place masked assignment
idx = torch.tensor([0, 2])
x.index_select(0, idx)     # gather rows; x[idx] is the same
```

2.5 Broadcasting

Broadcasting aligns shapes from the *trailing* dimension. Two dims are compatible when they are equal, or one of them is 1, or one is missing. The size-1 dim is virtually repeated—no memory is copied.

```
a = torch.randn(4, 3)          # (4, 3)
b = torch.randn(3)             # (3,)    -> treated as (1, 3) -> (4, 3)
c = a + b                      # (4, 3): the same row vector added everywhere
```

```
d = torch.randn(4, 1)          # (4, 1)  -> (4, 3)
e = a + d                      # (4, 3): a column vector added everywhere
f = a.mean(dim=0, keepdim=True) + a   # center the columns, still (4, 3)
```

PITFALL

Silently-wrong broadcasting is worse than an error: `(B, L, D)` plus `(L, D)` runs but broadcasts along the batch dim too, which is rarely what you mean. When in doubt, `unsqueeze` explicitly: `x + y.unsqueeze(0)`. A stray trailing 1 in the shape you meant to keep (`(B, L, 1)` vs `(B, L)`) is the classic symptom.

2.6 Reductions and the meaning of dim

Every reduction takes a `dim` (or tuple of dims): the operation *removes* the given dim, aggregating across it. `keepdim=True` keeps a size-1 dim so the result broadcasts back against the original.

```
x = torch.randn(2, 3, 4)
x.sum(dim=0).shape        # (3, 4): summed over the batch
x.sum(dim=-1).shape       # (2, 3): summed over the last dim
x.mean(dim=(1, 2)).shape  # (2,):   per-sample mean - common for normalization
x.sum(dim=1, keepdim=True).shape   # (2, 1, 4): broadcasts against x

x.argmax(dim=-1)          # index of the max per row - how predictions are read
x.max(dim=-1)             # named tuple: .values and .indices
(x > 0).float().mean()    # fraction of positive entries
```

PITFALL

Reading `dim` as "which dimension to walk along" causes subtle errors. Read it as "which dimension disappears." `x.sum(dim=0)` sums over the batch and leaves one value per feature; `x.sum(dim=1)` sums over features and leaves one value per sample. Matrix multiply `A @ B` contracts `A`'s last dim with `B`'s second-to-last: `(B, L, D) @ (D, V)` gives `(B, L, V)`.

CHAPTER 3

Autograd

Every PyTorch operation on tensors that have `requires_grad=True` is recorded into a dynamic computational graph. Calling `.backward()` on a scalar walks that graph in reverse and deposits gradients into each leaf tensor's `.grad`. The graph is rebuilt from scratch on every forward pass—that is the whole trick, and it is why control flow (loops, ifs) in your model code is free.

3.1 The basic cycle

```
import torch

w = torch.randn(3, requires_grad=True)     # a leaf we want gradients for
x = torch.randn(5, 3)

y = x @ w              # forward: nodes recorded as they run
loss = (y ** 2).mean()

loss.backward()        # reverse-mode autodiff through the graph
print(w.grad)          # d(loss)/dw, same shape as w
print(x.grad)          # None - x didn't require grad
```

Gradients *accumulate*: calling `backward()` twice adds to `.grad` rather than replacing it. That is why every training step clears them first—the real reason behind the mysterious `optimizer.zero_grad()` line:

```
loss.backward()
loss.backward()        # w.grad now holds the SUM of both backward passes
w.grad.zero_()         # manual clearing; optimizers do this for you
```

3.2 Disabling the graph

Three tools control graph construction, and choosing the right one matters:

- `torch.no_grad()` — the standard wrapper for evaluation and inference: no graph is built, memory and time are saved.
- `tensor.detach()` — a view of the tensor cut out of the graph; gradients do not flow through it. Essential in RL targets.
- `tensor.requires_grad_()` — flips the flag in place; how you mark inputs to be learned (or freeze parameters, below).

```
with torch.no_grad():              # evaluation - never forget this
    val_loss = loss_fn(model(xb), yb)

with torch.no_grad():
    y_target = model2(x).detach()  # .detach() inside a graph is also valid

@torch.no_grad()                   # decorator form
def predict(model, x):
```

```
    return model(x).argmax(dim=-1)
```

PITFALL

`with torch.no_grad():` must wrap evaluation loops. Without it you build a graph over the whole validation set, waste memory, and—if you then call `backward()`—silently pollute your training gradients. The twin rule in Chapter 6 is `model.eval()` for BatchNorm and Dropout; they go together.

3.3 Freezing parameters

Transfer learning (Ch. 7) is just autograd bookkeeping: exclude parameters from the graph and the optimizer never touches them.

```
model = load_pretrained()
for p in model.parameters():
    p.requires_grad_(False)          # freeze the backbone

for p in model.head.parameters():    # unfreeze only the new head
    p.requires_grad_(True)

trainable = [p for p in model.parameters() if p.requires_grad]
opt = torch.optim.AdamW(trainable, lr=1e-4)
```

3.4 Where the graph ends

Leaves are tensors you created (parameters, inputs); intermediates are everything computed. Two consequences worth memorizing:

- `.grad` is only ever populated on *leaves*. Intermediates keep a `.grad_fn` (their backward node) instead.
- `backward()` needs a scalar. For a non-scalar output `y`, supply the seed vector: `y.backward(gradient=g)` — rarely needed outside per-sample gradient work.

PITFALL

In RL and self-supervised training you compute a target with the current network and then regress the network toward it. The target must be built under `no_grad()` (or `.detach()`ed), or the gradient flows through the target too and training diverges. See the DQN loop in Ch. 10: `with torch.no_grad(): y = r + gamma * target(s2).max(...)`.

CHAPTER 4

Building Models: nn.Module

A model is a subclass of `nn.Module`. You declare layers in `__init__` and define data flow in `forward`. Everything else—parameter registration, moving to GPU, saving, train/eval switching—is inherited machinery worth understanding once.

4.1 The anatomy of a module

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class MLP(nn.Module):
    def __init__(self, d_in, d_hidden, n_classes):
        super().__init__()                      # ALWAYS first line
        self.net = nn.Sequential(
            nn.Linear(d_in, d_hidden),
            nn.ReLU(),
            nn.Linear(d_hidden, n_classes),     # raw logits, no softmax here
        )

    def forward(self, x):                       # x: (B, d_in)
        return self.net(x)                      # logits: (B, n_classes)

model = MLP(20, 64, 3)
logits = model(torch.randn(8, 20))              # __call__ runs forward
```

Assigning a layer (or another module) to an attribute of a `nn.Module` registers it: its parameters join `model.parameters()`, its buffers join `model.state_dict()`. That registration is the entire magic; there is nothing else to it.

```
for name, p in model.named_parameters():
    print(name, tuple(p.shape))     # net.0.weight, net.0.bias, ...

n_params = sum(p.numel() for p in model.parameters())
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)

model.to("cuda")                    # moves all params/buffers in place
model.train()                       # enable dropout/batchnorm training mode
model.eval()                        # switch them off for evaluation
```

4.2 The layer catalog

Layer	Shape contract and notes
`nn.Linear(D_in, D_out)`	`(..., D_in) -> (..., D_out)`; works on any leading dims
`nn.Conv2d(C_in, C_out, k)`	`(B, C_in, H, W) -> (B, C_out, H', W')`; channels FIRST
`nn.BatchNorm2d(C)`	normalizes each channel over batch+space; train/eval differ
`nn.LayerNorm(D)`	normalizes features per sample; the Transformer default
`nn.Dropout(p)`	zeroes a fraction p of activations during training only
`nn.Embedding(V, D)`	`(...)` int64 indices -> (..., D) learned vectors
`nn.LSTM(D, H, num_layers, batch_first=True)`	returns outputs and (h, c) states
`nn.MultiheadAttention(D, heads, batch_first=True)`	query, key, value -> attn out
`nn.TransformerEncoder(encoder_layer, n)`	stacks blocks; supply masks explicitly

PITFALL

`nn.Linear` takes `(in_features, out_features)` in that order; `nn.Conv2d` takes `(in_channels, out_channels, kernel)`. Getting them backwards usually surfaces as a cryptic shape error one layer later. Also: never apply `nn.Softmax` inside a model whose output feeds `nn.CrossEntropyLoss`—the loss applies log-softmax itself (Ch. 6).

4.3 Containers

```
seq = nn.Sequential(nn.Linear(8, 32), nn.ReLU(), nn.Linear(32, 4))

blocks = nn.ModuleList([Block(64) for _ in range(6)])   # registered list
for b in blocks:                    # iterate in forward; index like a list
    x = b(x)

heads = nn.ModuleDict({"cls": nn.Linear(64, 10), "reg": nn.Linear(64, 1)})
out = heads["cls"](x)
```

Use `nn.ModuleList`, never a plain Python list—a bare list is invisible to `parameters()` and its layers silently never train.

4.4 Initialization

PyTorch layers initialize sensibly by default (Kaiming-uniform for `Linear`/`Conv`). Override wholesale with `apply`, which recursively visits every submodule:

```
def init_weights(m):
    if isinstance(m, (nn.Linear, nn.Conv2d)):
        nn.init.kaiming_normal_(m.weight, nonlinearity="relu")
        if m.bias is not None:
```

```
            nn.init.zeros_(m.bias)
    elif isinstance(m, (nn.BatchNorm2d, nn.LayerNorm)):
        nn.init.ones_(m.weight)
        nn.init.zeros_(m.bias)

model.apply(init_weights)
```

4.5 Saving and loading

Always serialize the `state_dict` (an OrderedDict of tensors), never the module object—pickled modules break across code changes and devices.

```
torch.save(model.state_dict(), "model.pt")
model.load_state_dict(torch.load("model.pt", map_location=device))
model.eval()

# research checkpoints carry everything needed to RESUME
torch.save({
    "epoch": epoch,
    "model": model.state_dict(),
    "opt": optimizer.state_dict(),
    "sched": scheduler.state_dict(),
    "best_val": best_val,
}, "ckpt.pt")
ckpt = torch.load("ckpt.pt", map_location=device)
model.load_state_dict(ckpt["model"]); optimizer.load_state_dict(ckpt["opt"])
```

PITFALL

`load_state_dict` expects the same architecture and the same key names; a mismatch raises loudly (keep it that way—do not pass `strict=False` while debugging, it hides real errors). When loading on a CPU-only machine, use `map_location="cpu"` before `.to(device)`.

CHAPTER 5

Data Pipelines

PyTorch decouples *what* a sample is from *how* it is batched. A `Dataset` answers one question—given an index, return one sample. A `DataLoader` wraps it with batching, shuffling, and worker processes. You write the first once; the second is configuration.

5.1 The Dataset protocol

```
from torch.utils.data import Dataset, DataLoader, TensorDataset

# already-numeric data: two lines, no class needed
ds = TensorDataset(torch.randn(100, 8), torch.randint(0, 3, (100,)))
x0, y0 = ds[0]

# the general contract: __len__ + __getitem__
class MyDataset(Dataset):
    def __init__(self, xs, ys):
        self.xs, self.ys = xs, ys
    def __len__(self):
        return len(self.xs)
    def __getitem__(self, i):
        return self.xs[i], self.ys[i]
```

A realistic file-based dataset does its IO inside `__getitem__`, which is why workers help—each worker reads files in parallel:

```
from pathlib import Path
from PIL import Image

class ImageDataset(Dataset):
    """Directory layout: root/<class_name>/<image>.jpg"""
    def __init__(self, root, classes, transform=None):
        self.samples = [(p, yi) for yi, c in enumerate(classes)
                        for p in (Path(root) / c).glob("*.jpg")]
        self.transform = transform
    def __len__(self):
        return len(self.samples)
    def __getitem__(self, i):
        path, y = self.samples[i]
        x = Image.open(path).convert("RGB")
        if self.transform is not None:
            x = self.transform(x)        # PIL image -> tensor, plus augmentation
        return x, y
```

5.2 The DataLoader

```
loader = DataLoader(ds, batch_size=64, shuffle=True, num_workers=2,
                    pin_memory=True, drop_last=False)

for xb, yb in loader:                # xb: (B, ...), yb: (B,)
    ...                              # one optimization step per iteration
```

Option	Meaning
`shuffle=True`	reshuffle every epoch; use for *training only*
`num_workers`	parallel loader processes; 2–8 typical, 0 = inline
`pin_memory=True`	page-locked staging for faster host-to-GPU copies
`drop_last=True`	discard the final partial batch; matters with BatchNorm
`collate_fn`	custom per-batch assembly for variable-size samples

PITFALL

Validation and test loaders must use `shuffle=False`—shuffled evaluation makes metrics irreproducible and comparisons meaningless. `pin_memory=True` only pays off when the batch moves to CUDA; and `num_workers > 0` must stay at 0 inside notebook `__main__` on Windows and macOS spawn setups, or workers will crash-loop.

5.3 Transforms and augmentation

`torchvision` transforms compose; the same object usually applies to PIL images on the way in:

```
from torchvision import transforms as T

train_tf = T.Compose([
    T.RandomCrop(32, padding=4),     # augmentation: train only
    T.RandomHorizontalFlip(),
    T.ToTensor(),                    # PIL [0,255] -> float tensor [0,1], (C, H, W)
    T.Normalize(mean, std),          # channel-wise standardization
])
eval_tf = T.Compose([T.ToTensor(), T.Normalize(mean, std)])
```

The normalization constants are dataset-specific. Memorize the two standard ones; using them is part of reproducing reference results:

	mean	std
CIFAR-10	`(0.4914, 0.4822, 0.4465)`	`(0.2470, 0.2435, 0.2616)`
MNIST	`(0.1307,)`	`(0.3081,)`

PITFALL

`ToTensor()` divides by 255; if your data is already a float tensor in `[0, 1]`, do not re-apply it. Never put random augmentation in the validation/test transform—evaluate on uncorrupted data.

5.4 Train/val/test splits

```python
from torch.utils.data import Subset, random_split

n = len(full_ds)
n_train, n_val = int(0.8 * n), int(0.1 * n)
gen = torch.Generator().manual_seed(0)
train_ds, val_ds, test_ds = random_split(
    full_ds, [n_train, n_val, n - n_train - n_val], generator=gen)
```

When training and validation need *different transforms* (they should), split indices first and build two datasets over the same indices:

```python
idx = torch.randperm(n, generator=gen)
train_set = Subset(ImageDataset(root, classes, train_tf), idx[:n_train])
val_set   = Subset(ImageDataset(root, classes, eval_tf),  idx[n_train:])
```

PITFALL

`random_split` on a transform-carrying dataset gives train and val the same transform. And whatever the split mechanism, split *once with a fixed seed*—splitting with fresh randomness per run makes numbers incomparable.

5.5 Variable-length samples: collate_fn

The default collate stacks samples into a tensor, which fails when lengths differ (text, point clouds). Pad instead and hand the lengths to the model:

```python
from torch.nn.utils.rnn import pad_sequence

def collate(batch):
    seqs, labels = zip(*batch)                      # tuple of (L_i,) tensors
    lengths = torch.tensor([len(s) for s in seqs])
    x = pad_sequence(seqs, batch_first=True)        # (B, L_max), zero-padded
    return x, lengths, torch.tensor(labels)

loader = DataLoader(ds, batch_size=32, collate_fn=collate, shuffle=True)
```

CHAPTER 6

The Training Loop

Every model in this book—classifier, Transformer, RL agent—trains with the same five-step heartbeat: zero gradients, forward, loss, backward, step. This chapter is the canonical boilerplate; later chapters only vary the data and the loss.

6.1 The canonical template

One function serves both training and validation: pass an optimizer to train, `None` to evaluate.

```
import torch.nn as nn

def run_epoch(model, loader, loss_fn, opt=None, device="cpu"):
    training = opt is not None
    model.train(training)                # eval() when validating
    total, n = 0.0, 0
    for xb, yb in loader:
        xb = xb.to(device, non_blocking=True)
        yb = yb.to(device, non_blocking=True)

        with torch.set_grad_enabled(training):   # no graph when evaluating
            logits = model(xb)                   # forward
            loss = loss_fn(logits, yb)           # loss

        if training:
            opt.zero_grad(set_to_none=True)      # 1) clear old gradients
            loss.backward()                      # 2) backpropagate
            nn.utils.clip_grad_norm_(model.parameters(), 5.0)
            opt.step()                           # 3) update parameters

        total += loss.item() * xb.size(0)        # weighted by batch size
        n += xb.size(0)
    return total / n

for epoch in range(1, EPOCHS + 1):
    tr = run_epoch(model, train_loader, loss_fn, opt, device)
    va = run_epoch(model, val_loader, loss_fn, None, device)
    sched.step()                             # epoch-level schedulers only
    print(f"epoch {epoch:03d} | train {tr:.4f} | val {va:.4f}")

    if va < best_val:                        # keep the best model, not the last
        best_val = va
        torch.save({"model": model.state_dict(), "epoch": epoch}, "best.pt")
```

Memorize this shape. The subtle lines: `model.train(training)` flips Dropout and BatchNorm; `set_grad_enabled` saves the validation graph; the epoch loss is batch-size-weighted because the final partial batch is smaller; and you checkpoint on the best validation value, not the last epoch.

6.2 Loss functions

Task	Loss	Expects
Multiclass	`nn.CrossEntropyLoss`	raw logits `(B, C)`, labels `(B,)` int64
Binary/multilabel	`nn.BCEWithLogitsLoss`	logits `(B,)`, targets `(B,)` float
Regression	`nn.MSELoss` / `nn.L1Loss`	same-shape prediction and target
Robust regression	`nn.SmoothL1Loss` / `nn.HuberLoss`	same shape; less outlier-sensitive

PITFALL

`CrossEntropyLoss` = log-softmax + NLL. Passing it probabilities or applying `nn.Softmax` in the model double-squashes the logits and training stalls. The same "logits in" rule covers `BCEWithLogitsLoss`; use plain `BCELoss` only when you truly have probabilities (you almost never do). Labels for cross-entropy are class *indices*, not one-hot vectors.

6.3 Optimizers

Optimizer	Typical lr	When to reach for it
`SGD(momentum=0.9)`	0.01–0.1	CNNs; best final accuracy with a schedule; needs tuning
`Adam`	1e-4–3e-4	robust default for anything with attention or embeddings
`AdamW`	1e-4–3e-4	Adam with decoupled weight decay; the modern default everywhere

```
opt = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9,
                      weight_decay=5e-4, nesterov=True)
opt = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=0.01)
```

`weight_decay` is regularization via shrinking weights toward zero; keep it at `0.01` for AdamW, and consider excluding norms and biases from decay in larger projects.

6.4 Learning-rate schedules

Cosine decay to zero is the workhorse; warmup protects early training when Adam moments are still cold.

```
from torch.optim.lr_scheduler import CosineAnnealingLR, OneCycleLR, ReduceLROnPlateau

sched = CosineAnnealingLR(opt, T_max=EPOCHS)          # step once per EPOCH

sched = OneCycleLR(opt, max_lr=3e-4, total_steps=len(train_loader) * EPOCHS)
                                                       # step once per BATCH

sched = ReduceLROnPlateau(opt, factor=0.5, patience=3)
sched.step(val_loss)                                   # this one takes a metric
```

PITFALL

Match `.step()` frequency to the scheduler type: epoch-level schedulers (`CosineAnnealingLR`) step once per epoch, per-step schedulers (`OneCycleLR`) once per batch. Calling `scheduler.step()` *before* `optimizer.step()` skips a learning-rate update and emits a warning; keep the optimizer step first.

6.5 Gradient clipping and accumulation

Clipping caps the global gradient norm—cheap insurance, standard in RNNs, Transformers, and RL. Accumulation simulates a larger batch by summing gradients over several small forward/backward passes:

```
nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)   # before opt.step()

# accumulate over ACCUM mini-batches: scale the loss, step less often
loss = loss_fn(logits, yb) / ACCUM
loss.backward()
if (it + 1) % ACCUM == 0:
    opt.step()
    opt.zero_grad(set_to_none=True)
```

6.6 Mixed precision

Half-precision compute with a gradient scaler; roughly 2x speed and half the memory on modern GPUs. The `torch.amp` spelling is the current API:

```
scaler = torch.amp.GradScaler("cuda")

for xb, yb in loader:
    xb, yb = xb.to(device), yb.to(device)
    with torch.amp.autocast("cuda", dtype=torch.float16):
        logits = model(xb)
        loss = loss_fn(logits, yb)
    scaler.scale(loss).backward()          # scaled backward
    scaler.unscale_(opt)                   # only if clipping after backward
    nn.utils.clip_grad_norm_(model.parameters(), 5.0)
    scaler.step(opt)                       # skips steps with inf/nan grads
    scaler.update()
    opt.zero_grad(set_to_none=True)
```

6.7 Measuring accuracy

```
@torch.no_grad()
def accuracy(model, loader, device="cpu"):
    model.eval()
    correct = n = 0
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        pred = model(xb).argmax(dim=-1)    # logits -> class index
        correct += (pred == yb).sum().item()
        n += yb.numel()
    return correct / n
```

6.8 Early stopping

```
patience, bad = 10, 0
for epoch in range(EPOCHS):
    ...
    if va < best_val:
        best_val, bad = va, 0
        torch.save(model.state_dict(), "best.pt")
    else:
        bad += 1
        if bad >= patience:
            print(f"early stop at epoch {epoch}"); break
```

PITFALL

Validation loss can rise while validation accuracy still improves (the model is getting confident, not correct). When both are logged, early-stop on the metric you actually report. And never select your final model on the test set—that number is spent the moment you look at it.

CHAPTER 7

Worked Example: CIFAR-10

This chapter assembles Chapters 2–6 into a complete, standard training script: a small residual CNN trained on CIFAR-10 (10 classes, 32x32 color images). It is the template to copy for any supervised image project.

7.1 Data

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms as T

mean = (0.4914, 0.4822, 0.4465)
std  = (0.2470, 0.2435, 0.2616)

train_tf = T.Compose([
    T.RandomCrop(32, padding=4),          # augmentation: training only
    T.RandomHorizontalFlip(),
    T.ToTensor(),
    T.Normalize(mean, std),
])
eval_tf = T.Compose([T.ToTensor(), T.Normalize(mean, std)])

train_set = datasets.CIFAR10("data", train=True,  download=True,
                             transform=train_tf)
val_set   = datasets.CIFAR10("data", train=False, download=True,
                             transform=eval_tf)
```

7.2 A small ResNet

The essential architecture pattern of the last decade: stacked residual blocks `y = F(x) + x`, with strided convolutions to downsample and a global average pool before the classifier. Read it with Chapter 4 in mind.

```
class BasicBlock(nn.Module):
    def __init__(self, cin, cout, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(cin, cout, 3, stride=stride, padding=1, bias=False)
        self.bn1   = nn.BatchNorm2d(cout)
        self.conv2 = nn.Conv2d(cout, cout, 3, stride=1, padding=1, bias=False)
        self.bn2   = nn.BatchNorm2d(cout)
        self.short = nn.Sequential()               # identity path
        if stride != 1 or cin != cout:             # shapes differ: project
            self.short = nn.Sequential(
                nn.Conv2d(cin, cout, 1, stride=stride, bias=False),
                nn.BatchNorm2d(cout),
            )
```

```
    def forward(self, x):                          # x: (B, cin, H, W)
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return F.relu(out + self.short(x))         # the residual add

class SmallResNet(nn.Module):
    def __init__(self, n_classes=10, width=64):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(3, width, 3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(width), nn.ReLU(),
        )
        self.layer1 = nn.Sequential(BasicBlock(width, width),
                                    BasicBlock(width, width))
        self.layer2 = nn.Sequential(BasicBlock(width, 2 * width, stride=2),
                                    BasicBlock(2 * width, 2 * width))
        self.layer3 = nn.Sequential(BasicBlock(2 * width, 4 * width, stride=2),
                                    BasicBlock(4 * width, 4 * width))
        self.head = nn.Linear(4 * width, n_classes)

    def forward(self, x):                          # x: (B, 3, 32, 32)
        x = self.layer3(self.layer2(self.layer1(self.stem(x))))
        x = F.adaptive_avg_pool2d(x, 1).flatten(1)  # (B, 4*width)
        return self.head(x)                         # (B, 10) logits
```

Convolutions keep spatial layout; batch statistics stabilize training; the stride-2 blocks halve resolution while doubling channels. The skip connection is what makes deep stacks trainable—gradients flow through it unattenuated.

7.3 Training configuration

```
device = "cuda" if torch.cuda.is_available() else "cpu"
model  = SmallResNet().to(device)
loss_fn = nn.CrossEntropyLoss()
opt    = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9,
                         weight_decay=5e-4, nesterov=True)
sched  = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=EPOCHS)

train_loader = DataLoader(train_set, batch_size=256, shuffle=True,
                          num_workers=4, pin_memory=True, drop_last=True)
val_loader   = DataLoader(val_set, batch_size=512, shuffle=False,
                          num_workers=2, pin_memory=True)
```

Then run exactly the Chapter 6 loop (the `run_epoch` function, scheduler step per epoch, best-checkpoint on validation accuracy). This configuration reaches roughly 92–94% test accuracy in 60–100 epochs—about an hour on a single consumer GPU. On CPU, reduce to 10 epochs just to see the machinery work.

7.4 Transfer learning in five lines

For small datasets, start from ImageNet features instead of training from scratch—freeze the backbone, retrain a new head:

```python
from torchvision import models

model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
for p in model.parameters():
    p.requires_grad_(False)
model.fc = nn.Linear(model.fc.in_features, n_classes)   # new trainable head
model = model.to(device)
opt = torch.optim.AdamW(model.fc.parameters(), lr=3e-4)
```

PITFALL

Three classic failures here: (1) forgetting `T.Normalize`—the model trains but plateaus badly; (2) augmenting the validation set—your reported accuracy becomes a random number; (3) feeding `(B, H, W, C)` numpy-style images—PyTorch convolutions want `(B, C, H, W)`, so `T.ToTensor()` does the transposition and you should not undo it.

CHAPTER 8

Sequence Models: the LSTM

Before attention, sequences were processed by *recurrence*: carry a hidden state through the sequence, one token at a time. Transformers (Ch. 9) won on scale and parallelism, but LSTMs still earn their page: they excel on small data, stream naturally, run on tiny budgets, and you will meet them in older papers, in RL codebases, and inside production systems. This chapter gives you the working subset.

8.1 The recurrence interface

```
import torch
import torch.nn as nn

lstm = nn.LSTM(input_size=32, hidden_size=64, num_layers=2,
               batch_first=True)            # (B, L, D) in and out - ALWAYS pass this
x = torch.randn(8, 20, 32)                  # (B, L, D): batch, time, features
out, (h, c) = lstm(x)
out.shape                                   # (8, 20, 64): hidden state at every step
h.shape                                     # (2, 8, 64): (num_layers, B, H) final h
c.shape                                     # (2, 8, 64): cell memory, same layout
```

Read it as: `out` holds the hidden state after *every* timestep; `h` and `c` hold only the final ones, stacked per layer. For classification you want one vector per sequence — either the final hidden state of the top layer `h[-1]`, or a masked mean of `out`.

> **PITFALL**
>
> `nn.LSTM` defaults to `batch_first=False`, i.e. `(L, B, D)` — the same trap as `nn.Transformer`. Pass `batch_first=True` explicitly, or your batches are silently read as sequences and training just gets quietly worse.

8.2 Worked example: text classification

Classify variable-length token sequences with the Chapter 5 `collate_fn` (padding) and the Chapter 6 training loop:

```
class TextClassifier(nn.Module):
    def __init__(self, vocab, d_embed=64, d_hidden=128, n_classes=2):
        super().__init__()
        self.embed = nn.Embedding(vocab, d_embed, padding_idx=0)
        self.lstm = nn.LSTM(d_embed, d_hidden, num_layers=1,
                            batch_first=True, bidirectional=False)
        self.head = nn.Linear(d_hidden, n_classes)     # from the final state

    def forward(self, x, lengths):                     # x: (B, L) int64
        e = self.embed(x)                              # (B, L, D)
        out, (h, c) = self.lstm(e)                     # h[-1] is the top layer's
        return self.head(h[-1])                        # final state: (B, H)
```

```
model = TextClassifier(vocab=vocab_size).to(device)
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.AdamW(model.parameters(), lr=3e-4)

for epoch in range(EPOCHS):                          # the Chapter 6 loop
    for x, lengths, y in train_loader:               # from collate_fn (Ch. 5)
        x, y = x.to(device), y.to(device)
        logits = model(x, lengths)
        loss = loss_fn(logits, y)
        opt.zero_grad(set_to_none=True)
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 5.0)
        opt.step()
```

Note what the loop does *not* do: no special handling for padding at the output side, because classification reads only the final state. It matters on the input side instead — next section.

8.3 Packed sequences: don't run on padding

After `pad_sequence`, trailing zeros are fake timesteps; the LSTM will happily consume them and corrupt the final state of short sequences. `pack_padded_sequence` tells the RNN where each sequence really ends:

```
from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence

e = self.embed(x)                                    # (B, L, D)
packed = pack_padded_sequence(e, lengths.cpu(), batch_first=True,
                              enforce_sorted=False)
out, (h, c) = self.lstm(packed)                      # runs only real steps
out, _ = pad_packed_sequence(out, batch_first=True)  # back to (B, L, 2H)
```

With packing, `h[-1]` is the state at each sequence's true end. `lengths` must live on the CPU; `enforce_sorted=False` lets PyTorch do the sorting for you. Bidirectional LSTMs return two final states (`h[-2]` backward, `h[-1]` forward) — concatenate them and double the head's input width.

> **PITFALL**
>
> Two silent padding bugs: (1) mean-pooling `out` without masking — padded zeros drag every short sequence's average toward zero; divide by the true `lengths` instead: `(out * mask).sum(1) / lengths.unsqueeze(1)`. (2) Feeding `vocab_size` as `input_size` — the LSTM wants the *embedding* dimension, not the vocabulary size; the embedding layer sits in front precisely to make that dimension small.

8.4 Stateful RNNs and truncated BPTT

By default each batch starts from a zero state — fine for independent documents. For a single very long stream, carry the state across windows and detach it to cap the backward pass:

```
state = None
for x, _ in stream_batches:                          # consecutive windows
    out, state = lstm(x, state)
    h, c = state
    state = (h.detach(), c.detach())                 # cut the graph here
    ...                                              # loss.backward() on this window
```

Forgetting the `detach()` keeps the whole history in one graph — memory grows without bound and the backward pass spans the entire stream.

Situation	Reach for
Short sequences, small data, streaming, tiny hardware	LSTM — fewer parameters, constant memory per step
Long-range dependencies, large datasets, GPUs idle anyway	Transformer (Ch. 9) — parallel over L, attention reaches everywhere
Reinforcement learning with partial observability	LSTM policy cores — the state is an explicit memory

CHAPTER 9

Transformers

The Transformer is sequence processing with two ideas: *attention* (every position reads from every other position) and *residual blocks* with normalization. This chapter builds one from scratch, trains a tiny character-level GPT, then shows the Hugging Face route for real models.

9.1 Attention from scratch

Scaled dot-product attention: each token's query is compared against all keys, the scores are softmaxed into weights, and the values are mixed:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

```
import math
import torch
import torch.nn as nn

def attention(q, k, v, mask=None):            # q,k,v: (B, H, L, d_k)
    d_k = q.size(-1)
    scores = q @ k.transpose(-2, -1) / math.sqrt(d_k)   # (B, H, L, L)
    if mask is not None:                      # 1 = attend, 0 = blocked
        scores = scores.masked_fill(~mask, float("-inf"))
    w = scores.softmax(dim=-1)                # rows sum to 1
    return w @ v, w                           # (B, H, L, d_k), (B, H, L, L)

class MultiHeadSelfAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        assert d_model % n_heads == 0
        self.h, self.d_k = n_heads, d_model // n_heads
        self.qkv = nn.Linear(d_model, 3 * d_model, bias=False)
        self.proj = nn.Linear(d_model, d_model, bias=False)

    def forward(self, x, mask=None):          # x: (B, L, D)
        B, L, D = x.shape
        q, k, v = self.qkv(x).chunk(3, dim=-1)
        q, k, v = (t.view(B, L, self.h, self.d_k).transpose(1, 2)
                   for t in (q, k, v))        # (B, H, L, d_k) each
        out, w = attention(q, k, v, mask)
        out = out.transpose(1, 2).reshape(B, L, D)     # merge heads
        return self.proj(out)
```

The scaling by $\sqrt{d_k}$ keeps softmax inputs in a sane range; the mask uses `masked_fill` with `-inf` so masked positions get zero weight after the softmax.

9.2 The block, and stacking

Modern pre-norm block: attention and an MLP, each wrapped in a residual connection with LayerNorm applied *before* the sublayer:

```python
class Block(nn.Module):
    def __init__(self, d_model, n_heads, dropout=0.1):
        super().__init__()
        self.ln1, self.ln2 = nn.LayerNorm(d_model), nn.LayerNorm(d_model)
        self.attn = MultiHeadSelfAttention(d_model, n_heads)
        self.mlp = nn.Sequential(
            nn.Linear(d_model, 4 * d_model),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(4 * d_model, d_model),
        )

    def forward(self, x, mask=None):          # x: (B, L, D)
        x = x + self.attn(self.ln1(x), mask)  # residual around attention
        x = x + self.mlp(self.ln2(x))         # residual around MLP
        return x
```

For production use, `nn.TransformerEncoder` gives you the same thing battle-tested—but read its defaults first:

```python
layer = nn.TransformerEncoderLayer(d_model=256, nhead=4, dim_feedforward=1024,
                                   dropout=0.1, batch_first=True, norm_first=True)
encoder = nn.TransformerEncoder(layer, num_layers=6)
out = encoder(x, mask=causal_bool_mask)       # x: (B, L, D) with batch_first=True
```

PITFALL

`nn.Transformer` and friends default to `batch_first=False`, i.e. `(L, B, D)`—pass `batch_first=True` or your batches are silently interpreted as sequences. Masks: a causal mask is `(L, L)` boolean with `True` = allowed; padding masks are `(B, L)` with `True` = real token. And every attention softmax runs over `dim=-1`, the key axis.

9.3 A tiny character-level GPT

A language model: embed tokens, add positions, stack causal blocks, predict the next token. Train it on any text file; it learns to generate text in that style. Roughly 1M parameters, trainable on a GPU in minutes.

```python
class TinyGPT(nn.Module):
    def __init__(self, vocab, d_model=128, n_heads=4, n_layers=4,
                 block_size=256):
        super().__init__()
        self.block_size = block_size
        self.tok = nn.Embedding(vocab, d_model)
        self.pos = nn.Embedding(block_size, d_model)   # learned positions
        self.blocks = nn.ModuleList(
            Block(d_model, n_heads) for _ in range(n_layers))
        self.ln = nn.LayerNorm(d_model)
        self.head = nn.Linear(d_model, vocab, bias=False)

    def forward(self, idx):                    # idx: (B, L) int64
        B, L = idx.shape
        pos = torch.arange(L, device=idx.device)
        x = self.tok(idx) + self.pos(pos)      # (B, L, D)
```

```
        causal = torch.tril(torch.ones(L, L, dtype=torch.bool,
                                       device=idx.device))
        for block in self.blocks:
            x = block(x, causal)
        return self.head(self.ln(x))          # (B, L, vocab) logits
```

Training data: sliding windows over a character stream, target = the same window shifted by one.

```
text  = open("input.txt", encoding="utf-8").read()
chars = sorted(set(text))
stoi  = {c: i for i, c in enumerate(chars)}
itos  = {i: c for c, i in stoi.items()}
vocab = len(chars)
data  = torch.tensor([stoi[c] for c in text], dtype=torch.int64)

def get_batch(batch_size=32):
    ix = torch.randint(len(data) - BLOCK - 1, (batch_size,))
    x = torch.stack([data[i : i + BLOCK] for i in ix])
    y = torch.stack([data[i + 1 : i + BLOCK] for i in ix])
    return x.to(device), y.to(device)

model = TinyGPT(vocab).to(device)
opt = torch.optim.AdamW(model.parameters(), lr=3e-4)
loss_fn = nn.CrossEntropyLoss()

for step in range(1, 5001):
    x, y = get_batch()
    logits = model(x)
    loss = loss_fn(logits.reshape(-1, vocab), y.reshape(-1))
    opt.zero_grad(set_to_none=True)
    loss.backward()
    nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    opt.step()
    if step % 500 == 0:
        print(f"step {step} | loss {loss.item():.3f}")
```

FLATTEN-THEN-LOSS

Cross-entropy wants `(N, C)` logits and `(N,)` labels, so the per-position logits `(B, L, vocab)` are reshaped to `(B*L, vocab)`. This one line is the only "clever" part of LM training loops, and it appears in every language-model codebase.

Sampling autoregressively—predict, append, repeat—is the same loop that drives every LLM:

```
@torch.no_grad()
def sample(model, prompt="\n", n=300, temperature=0.8, top_k=None):
    model.eval()
    idx = torch.tensor([[stoi[c] for c in prompt]], device=next(
        model.parameters()).device)
    for _ in range(n):
        logits = model(idx[:, -model.block_size:])
        logits = logits[:, -1] / temperature          # last position only
        if top_k is not None:
            kth = logits.topk(top_k).values[..., -1, None]
```

```
            logits[logits < kth] = float("-inf")      # nucleus-lite
        probs = logits.softmax(dim=-1)
        nxt = torch.multinomial(probs, 1)
        idx = torch.cat([idx, nxt], dim=1)
    model.train()
    return "".join(itos[i] for i in idx[0].tolist())
```

9.4 Fine-tuning with Hugging Face

For real models you rarely train from scratch; you fine-tune a pretrained checkpoint. The `Trainer` wraps the Chapter 6 loop entirely:

```
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          Trainer, TrainingArguments)
from datasets import load_dataset

ds = load_dataset("imdb")
tok = AutoTokenizer.from_pretrained("distilbert-base-uncased")
enc = ds.map(lambda b: tok(b["text"], truncation=True, padding="max_length"),
             batched=True)

model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased", num_labels=2)

args = TrainingArguments("out", per_device_train_batch_size=32,
                         learning_rate=2e-5, num_train_epochs=2,
                         eval_strategy="epoch")
trainer = Trainer(model=model, args=args,
                  train_dataset=enc["train"].shuffle(seed=0).select(range(2000)),
                  eval_dataset=enc["test"].select(range=1000))
trainer.train()
```

PITFALL

Fine-tuning LRs are 10x smaller than from-scratch LRs—`2e-5`, not `3e-4`—and 2–4 epochs suffice. Large LR or long training destroys the pretrained features (catastrophic forgetting). Tokenizer and model must come from the *same* checkpoint name.

CHAPTER 10

Reinforcement Learning

In RL an agent acts in an environment; the environment returns rewards; the agent learns a policy that maximizes expected return. Three classic agents cover most of the space: *REINFORCE* (direct policy gradient), *DQN* (value-based, discrete actions), and *PPO* (the robust industry default). All three fit on one page each.

10.1 The environment API: Gymnasium

```
import gymnasium as gym

env = gym.make("CartPole-v1")            # balance a pole; +1 reward per step
obs, info = env.reset(seed=0)            # start an episode
print(env.observation_space)             # Box(-3.4, 3.4, (4,), float32)
print(env.action_space)                  # Discrete(2)

next_obs, reward, terminated, truncated, info = env.step(action)
done = terminated or truncated           # episode over either way
env.close()
```

- `terminated`: the MDP reached a terminal state (pole fell) — do not bootstrap from it.
- `truncated`: the time limit ran out — the state is still alive, and bootstrapping from it is correct.

> **PITFALL**
> Collapsing `terminated` and `truncated` into one `done` flag is the single most common RL bug: the value estimate leaks the artificial time-limit reward. Keep both flags; bootstrap only on `1 - terminated`.

10.2 REINFORCE: policy gradient

Sample whole episodes with the current policy, compute discounted returns, then push up the log-probability of actions in proportion to their return. Pure Monte-Carlo, on-policy, simplest thing that works.

```
import torch
import torch.nn as nn

env = gym.make("CartPole-v1")
device = "cuda" if torch.cuda.is_available() else "cpu"
policy = nn.Sequential(
    nn.Linear(env.observation_space.shape[0], 128), nn.ReLU(),
    nn.Linear(128, env.action_space.n),
).to(device)
opt = torch.optim.Adam(policy.parameters(), lr=1e-3)
gamma = 0.99

def run_episode():
    logps, rewards = [], []
    obs, _ = env.reset()
    done = False
    while not done:
```

```
        s = torch.as_tensor(obs, dtype=torch.float32, device=device)
        dist = torch.distributions.Categorical(logits=policy(s))
        action = dist.sample()
        logps.append(dist.log_prob(action))     # keep the graph attached
        obs, r, term, trunc, _ = env.step(action.item())
        rewards.append(r)
        done = term or trunc
    return logps, rewards

for episode in range(1, 3001):
    logps, rewards = run_episode()
    returns, G = [], 0.0
    for r in reversed(rewards):                 # discounted return to go
        G = r + gamma * G
        returns.insert(0, G)
    returns = torch.as_tensor(returns, device=device)
    returns = (returns - returns.mean()) / (returns.std() + 1e-8)  # baseline

    loss = -(torch.stack(logps) * returns).sum()
    opt.zero_grad()
    loss.backward()
    opt.step()
    if episode % 100 == 0:
        print(f"ep {episode} | return {sum(rewards):.0f}")
```

The loss is minus the policy-gradient estimator $\mathbb{E}[\sum_t \log \pi(a_t|s_t)\, G_t]$; the baseline subtraction does not change the expectation but slashes variance. CartPole reaches 500 steps within a few hundred episodes.

10.3 DQN: learning the value of actions

Learn $Q(s, a)$ = expected return from taking a in s, then act greedily. Four ingredients make it stable, and all four are standard:

1. **Replay buffer** — decorrelates samples.
2. **Target network** — a lagged copy provides the regression target.
3. **ϵ-greedy** — explore with probability ϵ, decay it.
4. **Huber loss + clipping** — tolerate TD outliers.

```
from collections import deque
import random

class QNet(nn.Module):
    def __init__(self, n_obs, n_act):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_obs, 128), nn.ReLU(),
            nn.Linear(128, 128), nn.ReLU(),
            nn.Linear(128, n_act),
        )
    def forward(self, x):
        return self.net(x)

class Replay:
    def __init__(self, cap):
        self.buf = deque(maxlen=cap)
```

```
    def push(self, *args):
        self.buf.append(args)
    def sample(self, n):
        return random.sample(self.buf, n)
    def __len__(self):
        return len(self.buf)
```

```
def optimize(model, target, buf, opt, gamma, batch=128, device="cpu"):
    if len(buf) < batch:
        return
    s, a, r, s2, d = zip(*buf.sample(batch))
    s  = torch.as_tensor(s,  dtype=torch.float32, device=device)
    a  = torch.as_tensor(a,  dtype=torch.int64,   device=device).view(-1, 1)
    r  = torch.as_tensor(r,  dtype=torch.float32, device=device).view(-1, 1)
    s2 = torch.as_tensor(s2, dtype=torch.float32, device=device)
    d  = torch.as_tensor(d,  dtype=torch.float32, device=device).view(-1, 1)

    q_sa = model(s).gather(1, a)                        # Q(s, a)
    with torch.no_grad():                               # target is FROZEN
        q_next = target(s2).max(dim=1, keepdim=True).values
        y = r + gamma * q_next * (1.0 - d)              # no bootstrap if done
    loss = nn.functional.smooth_l1_loss(q_sa, y)

    opt.zero_grad()
    loss.backward()
    nn.utils.clip_grad_norm_(model.parameters(), 10.0)
    opt.step()
```

```
env    = gym.make("CartPole-v1")
device = "cuda" if torch.cuda.is_available() else "cpu"
model  = QNet(env.observation_space.shape[0], env.action_space.n).to(device)
target = QNet(env.observation_space.shape[0], env.action_space.n).to(device)
target.load_state_dict(model.state_dict())              # hard-sync once
target.eval()
opt    = torch.optim.Adam(model.parameters(), lr=3e-4)
buf    = Replay(50_000)
eps, eps_min, eps_decay = 1.0, 0.05, 0.995

for ep in range(1, 601):
    obs, _ = env.reset()
    done, total = False, 0.0
    while not done:
        if random.random() < eps:                       # explore
            action = env.action_space.sample()
        else:
            with torch.no_grad():
                s = torch.as_tensor(obs, dtype=torch.float32,
                                    device=device)
                action = int(model(s).argmax().item())
        nxt, r, term, trunc, _ = env.step(action)
        buf.push(obs, action, r, nxt, term)   # store 'term', not done
        obs, done, total = nxt, term or trunc, total + r

        optimize(model, target, buf, opt, gamma=0.99, device=device)
```

```
    eps = max(eps_min, eps * eps_decay)        # decay per EPISODE
    if ep % 10 == 0:
        target.load_state_dict(model.state_dict())
    if ep % 25 == 0:
        print(f"ep {ep} | return {total:.0f} | eps {eps:.2f}")
```

PITFALL

DQN details that each silently break training: store `terminated` (not `term or trunc`) in the buffer so the target stays correct at time limits; keep `no_grad` around the max-operator target or the network chases itself; actions are `int64` indices for `gather`, observations are `float32`; update the target network every few hundred steps (hard) or with `target_param.data.mul_(1-tau).add_(param.data, alpha=tau)` (polyak). If loss explodes, lower lr to `1e-4` first.

10.4 Vectorized environments

PPO is designed for *batched* interaction: step many environments at once, learn from the concatenated transitions. Gymnasium wraps this in one object:

```
def make_env():
    return gym.make("CartPole-v1")

envs = gym.vector.SyncVectorEnv([make_env for _ in range(8)])
obs, info = envs.reset(seed=0)          # obs: (8, 4) - one row per env
obs, rew, term, trunc, info = envs.step(envs.action_space.sample())
obs.shape, rew.shape                    # ((8, 4), (8,))
```

Every tensor in the PPO code below simply gains a leading env axis. The one wrinkle: when an environment ends, `SyncVectorEnv` resets it for you — and exactly *when* the reset happens depends on the gymnasium version's `autoreset_mode` (the default changed in 1.0).

AUTORESET AND THE TERMINAL OBSERVATION

With modern gymnasium, a terminated env's final observation arrives via `info["final_observation"]`, and the env restarts on the next step. Keep storing `term` (not `term or trunc`) for the bootstrap target, as in the DQN section — the time limit must keep bootstrapping. If autoreset semantics get confusing, the single-env version below is unambiguous and trains CartPole in minutes anyway.

10.5 PPO, complete

The workhorse. Actor and critic stay separate for clarity; in larger projects they often share a trunk with two heads:

```
class Actor(nn.Module):
    def __init__(self, n_obs, n_act, d=64):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(n_obs, d), nn.Tanh(),
                                 nn.Linear(d, d), nn.Tanh(),
                                 nn.Linear(d, n_act))
    def forward(self, obs):
        return self.net(obs)                        # logits
```

```
class Critic(nn.Module):
    def __init__(self, n_obs, d=64):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(n_obs, d), nn.Tanh(),
                                 nn.Linear(d, d), nn.Tanh(),
                                 nn.Linear(d, 1))
    def forward(self, obs):
        return self.net(obs).squeeze(-1)           # V(s)
```

Collect a rollout under the current policy. Note `v_next`: the critic value of the *true* next state, computed before any episode reset — this is what makes time-limit (truncated) transitions bootstrap correctly:

```
def rollout(actor, critic, env, n_steps, device):
    obs, _ = env.reset()
    data = {"o": [], "a": [], "logp": [], "v": [], "r": [],
            "term": [], "vnext": []}
    for _ in range(n_steps):
        o = torch.as_tensor(obs, dtype=torch.float32, device=device)
        with torch.no_grad():                      # old policy = frozen data
            dist = torch.distributions.Categorical(logits=actor(o))
            act = dist.sample()
            logp, v = dist.log_prob(act), critic(o)
            nxt, r, term, trunc, _ = env.step(act.item())
            v_next = critic(torch.as_tensor(nxt, dtype=torch.float32,
                                            device=device))
        data["o"].append(o); data["a"].append(act)
        data["logp"].append(logp); data["v"].append(v)
        data["r"].append(torch.tensor(r, device=device))
        data["term"].append(torch.tensor(float(term), device=device))
        data["vnext"].append(v_next)
        obs = nxt
        if term or trunc:
            obs, _ = env.reset()
    return {k: torch.stack(t) for k, t in data.items()}
```

Advantages by GAE; `lam` trades bias against variance (1.0 = plain discounted TD residuals, 0.0 = Monte-Carlo):

```
def gae(rewards, values, v_next, terms, gamma=0.99, lam=0.95):
    adv, g = [], 0.0
    for t in reversed(range(len(rewards))):
        delta = rewards[t] + gamma * v_next[t] * (1 - terms[t]) - values[t]
        g = delta + gamma * lam * (1 - terms[t]) * g
        adv.insert(0, g)
    return adv
```

The update: several epochs of minibatch ascent on the clipped surrogate, plus value regression and an entropy bonus:

```
def ppo_update(actor, critic, opt, data, adv, ret,
               clip=0.2, epochs=4, mb_size=256):
    o, a, old_logp = data["o"], data["a"], data["logp"]
```

```
    for _ in range(epochs):
        for mb in torch.randperm(o.size(0)).split(mb_size):
            dist = torch.distributions.Categorical(logits=actor(o[mb]))
            ratio = (dist.log_prob(a[mb]) - old_logp[mb]).exp()
            clipped = torch.clamp(ratio, 1 - clip, 1 + clip) * adv[mb]
            pi_loss = -torch.min(ratio * adv[mb], clipped).mean()
            v_loss = ((critic(o[mb]) - ret[mb]) ** 2).mean()
            loss = pi_loss + 0.5 * v_loss - 0.01 * dist.entropy().mean()
            opt.zero_grad(set_to_none=True)
            loss.backward()
            nn.utils.clip_grad_norm_(actor.parameters(), 0.5)
            nn.utils.clip_grad_norm_(critic.parameters(), 0.5)
            opt.step()
```

```
env = gym.make("CartPole-v1")
device = "cuda" if torch.cuda.is_available() else "cpu"
actor  = Actor(env.observation_space.shape[0], env.action_space.n).to(device)
critic = Critic(env.observation_space.shape[0]).to(device)
opt = torch.optim.Adam(list(actor.parameters())
                       + list(critic.parameters()), lr=3e-4)

for update in range(1, 201):
    data = rollout(actor, critic, env, n_steps=2048, device=device)
    adv = torch.stack(gae(data["r"], data["v"], data["vnext"], data["term"]))
    adv = (adv - adv.mean()) / (adv.std() + 1e-8)
    ret = adv + data["v"]                          # value target, detached
    ppo_update(actor, critic, opt, data, adv, ret)

    if update % 20 == 0:                           # greedy evaluation
        with torch.no_grad():
            o, _ = env.reset(); total, done = 0.0, False
            while not done:
                logits = actor(torch.as_tensor(o, dtype=torch.float32,
                                               device=device))
                o, r, term, trunc, _ = env.step(logits.argmax().item())
                total, done = total + r, term or trunc
        print(f"update {update} | greedy return {total:.0f}")
```

The clamp is the entire idea: once the new policy disagrees with the old by more than 20% on a sample, extra improvement earns nothing, so one bad update cannot destroy the policy. The canonical hyperparameters are exactly the ones shown: lr `3e-4`, `clip=0.2`, 4 epochs, entropy `0.01`, a few thousand rollout steps per update. CartPole reaches the 500-step ceiling within a couple of minutes on CPU.

10.6 Continuous control

Swap the categorical distribution for a Gaussian; everything else in PPO stays identical. The mean is state-dependent, the standard deviation a learned (state-independent) parameter:

```
class GaussianActor(nn.Module):
    def __init__(self, n_obs, n_act, d=64):
        super().__init__()
        self.mu = nn.Sequential(nn.Linear(n_obs, d), nn.Tanh(),
                                nn.Linear(d, d), nn.Tanh(),
```

```
                                nn.Linear(d, n_act))
        self.log_std = nn.Parameter(-0.5 * torch.ones(n_act))

    def dist(self, obs):
        return torch.distributions.Normal(self.mu(obs), self.log_std.exp())
```

```
env = gym.make("Pendulum-v1")           # continuous torque in [-2, 2]
actor = GaussianActor(env.observation_space.shape[0],
                      env.action_space.shape[0]).to(device)

obs, _ = env.reset()
dist = actor.dist(torch.as_tensor(obs, dtype=torch.float32, device=device))
action = dist.sample()                                # (n_act,)
a = action.cpu().numpy().clip(env.action_space.low,
                              env.action_space.high)
obs, r, term, trunc, _ = env.step(a)
```

In `rollout` and `ppo_update`, replace every `Categorical` with `actor.dist(o)` — `.sample()`, `.log_prob()`, and `.entropy()` all exist with the same signatures. `Pendulum-v1` runs on stock gymnasium (no MuJoCo) and trains on CPU in minutes. For strictly bounded actions, the cleaner route is tanh-squashing the Gaussian before the environment sees it — see the PPO implementation in OpenAI's *Spinning Up* for the log-prob correction this requires.

10.7 Choosing an algorithm

Situation	Reach for	Why
Discrete actions, quick baseline	REINFORCE	five minutes to implement
Discrete actions, sample-efficient	DQN (+ extras)	reuses every sample
Continuous control	PPO	Gaussian policy, stable updates
Anything you will ship	PPO	robust to hyperparameters, parallelizable

For continuous actions, swap `Categorical` for `torch.distributions.Normal` with a learned (state-dependent) standard deviation; everything else in PPO stays identical.

CHAPTER 11

Experiments and Statistics

A PhD lives and dies on the credibility of its comparisons. This chapter is the smallest honest protocol: what a run is, how many runs make an experiment, when an improvement is real, and how to search hyperparameters without fooling yourself.

11.1 The unit of evidence

A *run* is one training execution: a config, a seed, final metrics. An *experiment* is N runs identical except for the seed. One run is an anecdote; the seed lottery regularly moves validation accuracy by more than the improvements you are hunting for.

```
import json
from dataclasses import asdict

def train_once(cfg, seed):          # your Chapter 6 loop, refactored
    seed_everything(seed)
    ...                             # build model/data from cfg, train, eval
    return {"val_acc": val_acc, "test_acc": test_acc}

run_dir = pathlib.Path("runs") / cfg.run_name
rows = []
for seed in range(5):               # five seeds is the honest minimum
    out = train_once(cfg, seed)
    out.update(seed=seed, **asdict(cfg))
    rows.append(out)
    (run_dir / f"seed{seed}.json").write_text(json.dumps(out, indent=2))
```

Aggregate into the number you will actually report:

```
import pandas as pd

df = pd.DataFrame([json.loads(p.read_text())
                   for p in run_dir.glob("seed*.json")])
m, s = df.test_acc.mean(), df.test_acc.std()
print(f"test acc {m:.2f} +- {s:.2f} (n={len(df)})")
```

Report mean ± std over seeds, and say which std it is: the spread across seeds (this), not the standard error of the mean. Five seeds is the floor; small deltas need more.

11.2 When is the improvement real?

The most common self-deception in the field: *"+1% mean, so my method works."* Work through the checklist instead:

- Compare on the same seeds, same split, same budget. A method compared against a baseline trained for fewer steps has compared nothing.
- Overlapping error bars do not prove "no difference"; separated bars do not prove "real difference." With per-seed numbers for both methods, test the *paired differences*:

```python
from scipy import stats

d = df_b.test_acc.values - df_a.test_acc.values   # same seeds, method b - a
t, p = stats.ttest_1samp(d, 0.0)
print(f"mean delta {d.mean():+.3f}, t={t:.2f}, p={p:.4f}")
```

- With 3 seeds, a p-value is theater — run 5–10 and report the delta with its spread; let the effect size carry the argument.
- Report the worst seed too. A method that wins on average but collapses on one seed out of five is telling you something.
- The test set answers once (Ch. 6). Model selection happens on validation; the test number is the last thing you compute, not a dashboard you check daily.

11.3 Ablation discipline

An ablation removes one component and measures the damage. The rules are strict because the temptation is not:

- Change *one* factor per experiment. Two changes and the result attributes to neither.
- The baseline must share everything else — data order, schedule, seeds, budget. A baseline from your last paper is not a baseline.
- Name runs by hypothesis, not by date: `cifar-norm-bn-vs-ln`, not `run17-final-v2-fixed`.

Keep a running results table in the project's README — it is the cheapest memory a lab has:

Variant	Test acc	Reading
baseline (Ch. 7)	93.1 ± 0.4	reference
− random crop/flip	91.0 ± 0.5	augmentation carries ~2 pts
− cosine schedule	92.6 ± 0.3	schedule is worth ~0.5 pts
− weight decay	92.2 ± 0.6	mild regularization helps

11.4 Hyperparameter sweeps with Optuna

Sweep *validation* performance, never test. Optuna wraps your Chapter 6 loop in an objective and searches it:

```python
import optuna

def objective(trial):
    cfg = Config(
        lr=trial.suggest_float("lr", 1e-5, 1e-2, log=True),
        batch_size=trial.suggest_categorical("batch_size", [128, 256]),
        weight_decay=trial.suggest_float("wd", 1e-6, 1e-1, log=True),
    )
    accs = [train_once(cfg, seed)["val_acc"] for seed in (0, 1, 2)]
    return sum(accs) / len(accs)

study = optuna.create_study(direction="maximize",
                            sampler=optuna.samplers.TPESampler(seed=0),
                            pruner=optuna.pruners.MedianPruner(n_warmup_steps=5))
study.optimize(objective, n_trials=50)
```

```
print(study.best_params, f"{study.best_value:.4f}")
```

- `TPESampler` models which regions of the space have paid off and samples there — usually better than grid or pure random at the same budget.
- `MedianPruner` kills trials that are worse than the median at the same epoch. It needs your loop to call `trial.report(value, step)` during training to act.
- Search `log=True` scales for quantities that span orders of magnitude (learning rate, weight decay). Linear search over lr wastes its budget between `1e-1` and `1e-2`.

PITFALL

The winner of a 50-trial sweep is the best of 50 *validation* estimates — it is optimistically biased by construction. Confirm by retraining the best config on fresh seeds and reading the numbers off the test set once. And do not sweep two correlated knobs independently (learning rate and batch size move together: scale lr with batch).

THE REPORTING CHECKLIST

Before a result leaves your laptop: number of seeds and their spread, total compute (GPU-hours), code commit hash, config files, and the date the test set was last touched (ideally: never before today). This one paragraph is what separates a table from a result.

CHAPTER 12

Research Craft

The difference between a script and a research tool: the second one is reproducible, resumable, and debuggable at 2am. This chapter is the minimal discipline around the training loop.

12.1 Reproducibility

```python
import os, random
import numpy as np
import torch

def seed_everything(seed=0):
    os.environ["PYTHONHASHSEED"] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

seed_everything(0)
torch.backends.cudnn.deterministic = True   # bitwise reproducible (slower)
torch.backends.cudnn.benchmark = False      # set True to autotune for speed
```

Full determinism also requires `num_workers=0` or worker seeding, and some ops have no deterministic kernel. In practice: seed everything, pin your seeds in the config, and treat runs as reproducible up to GPU nondeterminism unless you need bitwise equality.

12.2 Experiment hygiene

One directory per run, one config per directory, everything else regenerable:

```python
from dataclasses import dataclass, asdict
import json, pathlib

@dataclass
class Config:
    dataset: str = "cifar10"
    lr: float = 3e-4
    batch_size: int = 256
    epochs: int = 60
    seed: int = 0
    run_name: str = "resnet-cosine"

cfg = Config()
run_dir = pathlib.Path("runs") / cfg.run_name
run_dir.mkdir(parents=True, exist_ok=True)
(run_dir / "config.json").write_text(json.dumps(asdict(cfg), indent=2))
```

The checkpoint format from Chapter 4 already carries model, optimizer, scheduler, and epoch—*resume* is then loading that dict and continuing the loop. Checkpoint at least as often as your cluster's job-time limit.

12.3 Logging

Start with print. Graduate to TensorBoard the day you compare two runs:

```
from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter(run_dir)

writer.add_scalar("train/loss", train_loss, global_step)
writer.add_scalar("val/loss", val_loss, global_step)
writer.add_scalar("val/acc", acc, global_step)
writer.add_scalar("opt/lr", opt.param_groups[0]["lr"], global_step)
# visualize once, early:
writer.add_image("samples", xb[:8], global_step)
```

View with `tensorboard --logdir runs`. Weights & Biases or MLflow are drop-in upgrades if you need cross-machine dashboards; the scalars you log barely change.

12.4 Speed checklist

Lever	Typical effect
`num_workers=4-8`, `pin_memory=True`	removes input-bound stalls
Mixed precision (Ch. 6)	~2x step time and half the memory on modern GPUs
Larger batch + higher lr	throughput until you hit memory
`torch.compile(model)`	free 10–30% after a slow first step
`set_to_none=True` in `zero_grad`	skips a needless memset
Profile before optimizing: `torch.profiler`	measures, rather than guesses

12.5 Multi-GPU, complete

One process per GPU; each rank sees a disjoint shard of the data; gradients are averaged automatically. The whole dance fits in one script:

```
import os
import torch
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.utils.data.distributed import DistributedSampler

def ddp_setup():
    dist.init_process_group("nccl")            # "gloo" for CPU debugging
    torch.cuda.set_device(int(os.environ["LOCAL_RANK"]))

def main():
    ddp_setup()
    rank, world = dist.get_rank(), dist.get_world_size()
    device = torch.device(f"cuda:{rank}")

    model = SmallResNet().to(device)           # any Chapter 4 model
    ddp_model = DDP(model)                     # wrap AFTER .to(device)
    opt = torch.optim.AdamW(ddp_model.parameters(), lr=3e-4)
```

```
    sampler = DistributedSampler(train_set, num_replicas=world,
                                 rank=rank, shuffle=True)
    loader = DataLoader(train_set, batch_size=256, sampler=sampler,
                        num_workers=4, pin_memory=True)

    for epoch in range(EPOCHS):
        sampler.set_epoch(epoch)               # reshuffle shards per epoch
        tr = run_epoch(ddp_model, loader, loss_fn, opt, device)  # Ch. 6 loop
        if rank == 0:                          # exactly one writer
            print(f"epoch {epoch} | loss {tr:.4f}")
            torch.save(model.state_dict(), "ckpt.pt")   # unwrapped weights
    dist.destroy_process_group()

if __name__ == "__main__":
    main()
```

```
torchrun --nproc_per_node=4 train.py
```

- The effective batch size is `per_rank_batch * world` — scale the learning rate up accordingly (Ch. 11).
- `run_epoch` is unchanged: DDP averages gradients inside `backward()`. Only logging, evaluation, and saving belong on rank 0.
- Save the *unwrapped* `model.state_dict()` — `ddp_model.state_dict()` prefixes every key with `module.` and will not load into a plain model later.
- The global training loss differs per rank (each sees its own shard); `dist.all_reduce(loss)` if you want the true mean.

12.6 Speed and memory

Measure before optimizing — the profiler tells you whether you are input-bound, launch-bound, or compute-bound:

```
from torch.profiler import profile, ProfilerActivity

with profile(activities=[ProfilerActivity.CPU,
                         ProfilerActivity.CUDA]) as prof:
    run_epoch(model, train_loader, loss_fn, opt, device)
print(prof.key_averages().table(sort_by="cuda_time_total", row_limit=12))
```

Then, in rough order of payoff:

Tool	What it does
`torch.compile(model)`	fuses elementwise chains and cuts kernel launches; 10–30% after a one-off compile wait
bf16 autocast	`autocast("cuda", dtype=torch.bfloat16)`: no `GradScaler` needed on Ampere or newer; fp16 keeps Ch. 6's scaler recipe
`checkpoint()`	rematerializes activations in backward — ~30% memory for ~30% extra compute
gradient accumulation	simulates big batches on small cards (Ch. 6)
`expandable_segments`	`PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` against fragmentation

Activation memory grows with `batch_size * seq_len * layers` — when a Transformer OOMs, halve the batch and accumulate (Ch. 6), or checkpoint the blocks:

```
from torch.utils.checkpoint import checkpoint

class DeepTransformer(nn.Module):
    def __init__(self, d_model, n_heads, n_layers):
        super().__init__()
        self.blocks = nn.ModuleList(
            Block(d_model, n_heads) for _ in range(n_layers))

    def forward(self, x, mask):
        for block in self.blocks:
            x = checkpoint(block, x, mask, use_reentrant=False)
        return x
```

PITFALL

`checkpoint(block, x, mask, ...)` replays `block` during backward — the inputs it saves must not be mutated in place, and any tensor that must stay fixed (like a mask) is passed as an argument, not captured in a closure. `use_reentrant=False` is the modern mode and works with `torch.compile` and DDP without extra arguments.

12.7 The debugging checklist

When training misbehaves, work down this list in order:

1. **Overfit one batch.** Take 32 samples and drive training loss to zero. If you cannot, the bug is in your code, not your hyperparameters.
2. **Look at the data.** Decode a few inputs back to pixels/text; check label ranges and dtype (`int64` for cross-entropy).
3. **Check shapes and ranges.** Print `x.shape`, `x.min()`, `x.max()` once per pipeline stage.
4. **Loss at init.** Multiclass loss should start near $\ln(C)$ (e.g. 2.30 for 10 classes). Wildly off means double softmax or wrong label format.
5. **Learning rate.** NaN loss: lower lr, check for `log(0)` or `sqrt(0)` in custom losses, ensure AMP scaling. No movement: raise it 10x.
6. **eval mode and no_grad** in validation—always both.
7. **Gradient flow.** After a step, confirm `p.grad is not None` for every trainable parameter; a silent `detach()` or frozen submodule breaks the chain.

12.8 Where to go next

Resource	What it is for
Official PyTorch tutorials	canonical coverage of every API touched here
Dive into Deep Learning (d2l.ai)	the theory behind each block, with runnable code
Karpathy, *Neural Networks: Zero to Hero*	builds GPT from raw autograd upward
nanoGPT repository	production-grade version of Ch. 9's tiny model
OpenAI *Spinning Up in Deep RL*	the theory + reference implementations of Ch. 10
CleanRL repository	single-file RL agents; compare against yours
Hugging Face course	Transformers, datasets, fine-tuning at scale

> **THE ONE HABIT**
> Write the run's config to disk, seed everything, and log scalars. Those three lines turn "I think lr 3e-4 worked better" into a table—which is the unit of progress in a PhD.

APPENDIX A

Cheat Sheet

The whole book compressed to API one-liners. Skim it after Chapter 6; return to it whenever a shape or a name slips your mind.

A.1 Tensors

Task	Code
Create from data	`torch.tensor([[1., 2.], [3., 4.]])`
Random	`torch.rand(2, 3)`, `torch.randn(2, 3)`, `torch.randint(0, 9, (2, 3))`
Constants / like	`torch.zeros(3, 4)`, `torch.ones_like(x)`, `torch.randn_like(x)`
Ranges	`torch.arange(0, 10, 2)`, `torch.linspace(0, 1, 5)`
Inspect	`x.shape`, `x.dtype`, `x.device`, `x.numel()`
Reshape	`x.reshape(2, 12)`, `x.flatten(1)`, `x.unsqueeze(0)`, `x.squeeze()`
Reorder	`x.transpose(1, 2)`, `x.permute(2, 0, 1)`
Join / split	`torch.cat([a, b], 0)`, `torch.stack([a, b])`, `x.chunk(3, dim=-1)`
Move / cast	`x.to("cuda", torch.float32)`, `x.long()`, `x.float()`
Reduce	`x.sum(0)`, `x.mean(-1, keepdim=True)`, `x.argmax(-1)`, `x.max(-1)`
Elementwise	`x + y`, `x * y`, `x @ y`, `torch.einsum("bld,dv->blv", x, w)`
Mask	`x[x > 0]`, `scores.masked_fill(~m, float("-inf"))`
To Python	`x.item()`, `x.tolist()`, `x.detach().cpu().numpy()`

A.2 Autograd

Task	Code
Track gradients	`w = torch.randn(3, requires_grad=True)`
Backward	`loss.backward()` then read `w.grad`
Clear	`opt.zero_grad(set_to_none=True)` or `w.grad.zero_()`
No graph	`with torch.no_grad(): ...` or `@torch.no_grad()`
Cut from graph	`y = x.detach()`
Freeze	`p.requires_grad_(False)`

A.3 Models

Task	Code
Define	subclass `nn.Module`: layers in `__init__`, dataflow in `forward`
Common blocks	`nn.Linear(D, H)`, `nn.Conv2d(C, C2, k)`, `nn.BatchNorm2d(C)`, `nn.LayerNorm(D)`, `nn.Dropout(p)`, `nn.Embedding(V, D)`
Containers	`nn.Sequential(...)`, `nn.ModuleList([...])`
Modes	`model.train()` / `model.eval()` — always pair with eval loops
Device	`model.to(device)`
Count	`sum(p.numel() for p in model.parameters())`
Save / load	`torch.save(model.state_dict(), "m.pt")` / `model.load_state_dict(torch.load("m.pt", map_location=dev))`
Predict	`model(x).argmax(dim=-1)`
Sequence	`nn.LSTM(D, H, batch_first=True)`: (B, L, D) -> (B, L, H) + (h, c); `h[-1]` is the final state

A.4 Data and training

Task	Code
Dataset	implement `__len__` + `__getitem__`
Loader	`DataLoader(ds, batch_size=64, shuffle=True, num_workers=4, pin_memory=True)`
Vision	`T.Compose([..., T.ToTensor(), T.Normalize(mean, std)])`
Losses	`nn.CrossEntropyLoss` (logits + int64 labels), `nn.BCEWithLogitsLoss`, `nn.MSELoss`
Optimizers	`AdamW(model.parameters(), lr=3e-4)`, `SGD(lr=0.1, momentum=0.9, nesterov=True)`
Schedule	`CosineAnnealingLR(opt, T_max=E)` (per epoch), `OneCycleLR(...)` (per batch)
Clip	`nn.utils.clip_grad_norm_(model.parameters(), 5.0)`
AMP	`with torch.amp.autocast("cuda"): ...` + `GradScaler("cuda")`
Checkpoint	save dict with `model`, `opt`, `epoch`
Seed	`random` / `numpy` / `torch` seeds + `torch.cuda.manual_seed_all`

A.5 RL with Gymnasium

Task	Code
Env	`env = gym.make("CartPole-v1")`; `obs, info = env.reset(seed=s)`
Step	`obs, r, term, trunc, info = env.step(a)`; episode over when `term or trunc`
Spaces	`env.observation_space.shape`, `env.action_space.n`
Policy dist	`Categorical(logits=model(s))`, then `.sample()`, `.log_prob(a)`, `.entropy()`
Vector envs	`gym.vector.SyncVectorEnv([...])`: every tensor gains a leading env axis
Continuous	`Normal(mu(obs), log_std.exp())` actor; clip actions to the space bounds
DQN target	`with torch.no_grad(): y = r + g * target(s2).max(1, keepdim=True).values * (1 - term)`
Polyak	`tp.data.mul_(1 - tau).add_(p.data, alpha=tau)`

A.6 Idioms worth memorizing

```
logits.reshape(-1, vocab), labels.reshape(-1)      # LM loss flattening
F.adaptive_avg_pool2d(x, 1).flatten(1)             # (B, C, H, W) -> (B, C)
nn.utils.rnn.pad_sequence(seqs, batch_first=True)  # variable-length batch
pack_padded_sequence(e, lengths.cpu(), batch_first=True)  # skip pad steps
torch.tril(torch.ones(L, L, dtype=torch.bool))     # causal mask
q @ k.transpose(-2, -1) / math.sqrt(d_k)           # attention scores
(x - x.mean()) / (x.std() + 1e-8)                  # normalize returns
torch.as_tensor(np_array, dtype=torch.float32)     # numpy -> tensor
```

A.7 The book at a glance

Ch.	Topic	One-line essence
1	Orientation	four abstractions; the loop is the recurring pattern
2	Tensors	shape, dtype, device; `dim` is the dimension that disappears
3	Autograd	graph built on forward, walked on backward, cleared every step
4	`nn.Module`	declaring a layer registers its parameters; `state_dict` is the contract
5	Data	`Dataset` says what a sample is; `DataLoader` says how it is batched
6	Training loop	zero, forward, loss, backward, step — checkpoint on validation
7	CIFAR-10	data + small ResNet + the Chapter 6 loop = a supervised project
8	Sequence models	recurrence before attention; pack padded batches; detach carried states
9	Transformers	attention + pre-norm blocks + next-token prediction; feed logits
10	RL	sample with the policy, regress toward returns; keep `term`/`trunc` apart
11	Experiments	mean ± std over seeds; ablate one factor; sweep on validation
12	Research craft	seed, log, checkpoint — reproducibility is the unit of progress

FINAL NOTE

The loop in Chapter 6 is the same everywhere; what changes is the data, the loss, and—in RL—returns standing in for labels. When a result looks too good, rerun with a new seed; when it looks impossible, check the shapes. Write the config to disk next to the checkpoint, and the experiment is someone else's to reproduce—most often, your own.